

Turn Salesforce + Slack into a self-running deal-intelligence machine.

The repeatable method behind every system I shipped at League — distilled to the moves that transfer. Tool-agnostic, anchored on the two systems Intradiem already runs.

Dallas Andrews · GTM
Engineer

Intradiem · Day 1: July 6,
2026

Stack: Salesforce +
Slack

THE THESIS

What a GTM Engineer actually does

Every B2B GTM org has the same shape: **Salesforce is the database, Slack is where reps actually live**, and a layer of point tools sits in between. What's almost always missing is the **middleware** — the thing that reads what happened, structures it, routes it to the right people, and writes it back to the CRM without a human retyping anything.

A GTM Engineer finds that missing layer, proves the gap is real, and closes it — fast, with the tools already in the building, before asking for new budget.

THE ARCHITECTURAL CONVICTION

Salesforce is the system of record. Slack is the interface. AI is the middleware that reads from one and writes to the other — so a rep has **near-zero reason to open Salesforce** during their day.

THE OPERATING LOOP

Seven moves, every build, in order

1

Diagnose before you build

Audit what exists before writing a line of code. Most failures are "built on broken data" failures. Three layers: the **connection** (is SF↔Slack wired and user-mapped?), the **object** (do the fields the AI writes to exist?), the **data** (deduped contacts, correct emails, populated roles?).

2

Find the missing middleware

Name the manual gap out loud. The pattern: a tool posts intelligence into Slack, humans read it, then retype it into Salesforce — or skip it, so the CRM rots. Three quick questions and you've located the build.

3

Lock the foundation

Before tooling, write down how the org's data is interpreted: record types, the revenue source-of-truth field, the real stage list, the hygiene risks. A persistent **Foundational Rules** file every AI session loads — so every tool shares one definition of "correct."

4

Architect in four layers

Decompose every build into Capture → Intelligence → Distribution → Write-back, then map each to a tool that exists or a thin piece you build. The layering is the portable part — the vendor never matters.

5

Engineer the intelligence

The "magic" is a prompt, not a model. **Context-enriched** (transcript + SF object context, not one-pass) and **stage-aware** (a different schema-enforced prompt per pipeline stage). Short and actionable beats a wall of text.

6

Close the loop

Most DIY builds stop at "posted to Slack." Don't. The differentiator is the write-back — activity record, next-steps, contact intelligence, qualification fields — automatic or two-click. This is what makes Slack a headless front end for Salesforce.

7

Govern, enforce, roll out

A system that works Day 1 and rots by Day 90 is a failure. Validation rules, dedupe rules, monitoring for OAuth/token failures. Then roll out in order: **connections first** (the most-skipped, most-fatal step), clean data before launch, 30 days of tuning.

THE ARCHITECTURE LENS

Every intelligence build is the same four layers

Layer	Job	Build with what's already there
1. Capture	Get a speaker-attributed transcript / signal somewhere readable	Existing recorder, or any tool that outputs a transcript
2. Intelligence	Transcript + CRM context → structured output	Claude API + a stage-aware prompt
3. Distribution	Route the structured output to the right Slack channel(s)	Slack API / webhook
4. Write-back	Push the structured fields into Salesforce	SF REST API, or a lightweight Slack→SF tool

The prompt is the product • Rule 1

Context-enriched, not one-pass. Feed the model the transcript plus the SF object context — stage, ARR, account, existing next steps. That's what produces source attribution and stage awareness instead of generic summary.

The prompt is the product • Rule 2

Stage-aware schema. A different extraction prompt per stage, each returning a defined JSON schema. A late-stage call focuses on legal/PO/signatures; an early call on pain + stakeholders.

APPLICATION

Day 1 → Week 1 at a Salesforce + Slack org**DAY 1 • ACCESS + AUDIT**

Get SF read access; run discovery queries (record types, stages, owners, revenue field, custom fields). Confirm the SF↔Slack connector is live and my own user is mapped. Map the stack in writing and name the middleware gap publicly — a specific written artifact, not a verbal claim.

DAYS 2-3 · FOUNDATION

Fill the Salesforce Foundational Rules file for this org. Audit the object layer (do next-steps / call-date / qualification fields exist? are Contact Roles enabled?). Run a contact email + duplicate audit on open-pipeline accounts.

DAYS 4-5 · FIRST PROOF

Stand up the smallest end-to-end slice that demonstrates the loop: capture → intelligence → Slack post → SF write-back, scoped to my own calls first. A live "call happened → Slack post → SF logged itself" demo beats any deck.

THROUGHOUT · PARTNER, DON'T LONE-WOLF

Find the RevOps/Salesforce admin and the Slack admin early — nothing routes or writes without them. Frame every build as closing a gap they already feel.

THE WALK-IN KIT

Day-1 diagnostic questions

STACK & GAP

- What conversation tool are you using, and does it connect to SF or just Slack?
- How do reps update Salesforce after a call — manual or automated?
- Do managers read Slack or SF for deal intel?
- Any Slack→SF automation running today?

FOUNDATION

- What fields are reps required to fill?
- Is there a qualification framework with real SF fields behind it?
- Deal-room channels in Slack + a naming convention?
- Who owns the SF config and who owns Slack?

HYGIENE & ACCESS

- Is SF↔Slack user-mapped cleanly by email/SAML?
- Is API access enabled on the integration user?
- Any security boundaries to respect (data-crossing, PII)?
- Where do duplicates and free-text fields live?

OPERATING BELIEFS

The principles underneath the method

- **The gap is the write-back, not the capture.** Capturing intelligence is easy and already happening; getting it back into the system of record automatically is the unsolved part.
- **The prompt is the product.** Stage-aware, context-enriched, schema-enforced. That's the leverage.
- **Design for failure.** Tokens expire, users get unmapped, data drifts. Monitoring and validation rules separate a demo from a system.
- **Ship the path of least resistance first.** A Slack webhook ships in hours; a fully-approved CRM integration takes weeks. Get value flowing, then close the loop.
- **Consistency comes from a shared knowledge layer,** not each tool's individual config. One Foundational Rules file beats five tools guessing.
- **Adoption is an ordering problem.** Connections first, clean data before launch, 30 days of tuning. Skip the order and the best architecture sits idle.

EVIDENCE

Proof artifacts behind the method

momentum-architecture-and-diy-guide.md

— Full reverse-engineering of a production call-intelligence stack and a layer-by-layer plan to replicate it with Claude API + Slack API + SF REST API at near-zero new cost.

call-recording-slack-bridge-pattern.md

— The portable recording→Slack→CRM bridge: channel architecture, exact message schema, qualification scoring taxonomy, and where the manual gap sits. Real production post examples included.

day1-prerequisites-sf-slack-setup.md

— The full Day-1 audit: SF↔Slack native connector setup, the object/field layer the AI writes to, contact-data hygiene fixes, security blockers to expect, complete pre-build checklist.

SALESFORCE_FOUNDATIONAL_TEMPLATE.pdf

— The reusable knowledge-layer template: record-type filters, currency/revenue source-of-truth, stage definitions, and the discovery queries to run before building anything.

The GTM Engineer Operating System · Extracted from builds shipped at League · Tool-agnostic, anchored on Salesforce + Slack

Dallas Andrews // GTM Engineer, Intradiem // Day 1: July 6, 2026